

Table des matières

Table des matières.....	2
Table de figures.....	5
Introduction Générale.....	6
Partie I: PHASE D'ANALYSE.....	7
I. Présentation générale du projet.....	7
II. Description du contexte.....	8
III. Problématique.....	10
IV. Définition des besoins.....	11
A. Besoins fonctionnels.....	11
B. Besoins non fonctionnels.....	12
C. Objectif général.....	13
V. Résultat attendu.....	14
VI. Démarche d'intervention.....	15
A. Architecture(Clean Architecture).....	15
1. Entities (Entités).....	15
2. Use Cases (Cas d'Utilisation).....	15
3. Interface Adapters (Adaptateurs d'Interface).....	16
4. Frameworks & Drivers (Cadres et Infrastructure).....	17
B. Planification.....	17
1. Analyse et Conception.....	17
i. Analyse des Besoins.....	17
ii. Conception.....	17
2. Phase Developpement.....	18
i. Configuration de l'Environnement.....	18
ii. Développement des Modules.....	18
3. Phase de Développement.....	18
i. Préparation au Déploiement.....	18
ii. Déploiement.....	19
VII. Outil collaboratif.....	20
Conclusion.....	21
Partie II: PHASE DE CONCEPTION DU PROJET.....	22
I. Architecture du projet.....	23
II. Conception fonctionnelle.....	23
1. Modélisation détaillée (UML).....	23
a. Diagramme de contexte.....	23
b. Diagramme de package.....	24
c. Diagramme de classes.....	24
2. Interface utilisateur (UI).....	25

III. Conception technique.....	25
1. Choix des technologies et justifications.....	26
a. Base de données.....	26
b. Langages de Programmatons.....	26
c. Frameworks.....	26
Partie III: API DE GEOFENCE.....	28
1. Introduction.....	28
2. Architecture du Système.....	28
2.1 Technologies Utilisées.....	28
2.2 Structure du Projet.....	28
3. Modèles de Données.....	30
3.1 Entités Principales.....	30
4. Fonctionnalités Principales.....	31
4.1 Gestion des Zones Géographiques.....	31
4.2 Suivi des Véhicules.....	31
4.3 Système d'Alertes.....	31
4.4 Notifications en Temps Réel.....	31
5. Diagramme de séquence.....	32
6. Algorithmes Clés.....	34
6.1 Calcul de Distance.....	34
6.2 Détection de Franchissement de Zone.....	34
7. Points Forts et Innovations.....	36
7.1 Système d'Alertes Flexible.....	36
7.2 Architecture Temps Réel.....	36
7.3 Gestion des Utilisateurs Multi-rôles.....	36
8. Améliorations Possibles.....	36
8.1 Optimisations de Performance.....	36
8.2 Fonctionnalités Additionnelles.....	36
8.3 Sécurité.....	36
9. Conclusion.....	37
Partie IV: GUIDE DE DÉPLOIEMENT.....	38
I. Introduction.....	38
1. Objectif du guide.....	38
2. Prérequis :.....	39
3. Gestion de compte et des accès :.....	39
II. Environnement de déploiement.....	40
1. Configuration système requise.....	40
2. Pré-requis logiciels.....	40
III. Installation du système.....	41

1. Cloning the GitHub Repository.....	41
2. Initializing the Application Locally.....	41
3. Verifying the Local Setup.....	41
Recap.....	42
IV. Déploiement de l'application.....	43
1. Procédure de déploiement.....	43
3. Tests de vérification.....	44
V. Dépannage (Troubleshooting).....	44
1. Contacter l'équipe Easy-Rent.....	44
2. Assistance GitHub.....	44
3. Assistance Vercel.....	44
4. Autres ressources utiles.....	44
Conclusion Générale.....	45

Table de figures

Figure 1: Architecture du projet Next JS.....	31
Figure 2: Diagramme de contexte.....	23
Figure 3: Diagramme de classe.....	24
Figure 4: Architecture de l'API de Geofence.....	28
Figure 5: Diagramme de classe - Geofence.....	29
Figure 6: Diagramme de séquence - Geofence.....	31

Introduction Générale

Dans un monde où la mobilité est essentielle, les entreprises de location de véhicules jouent un rôle crucial en facilitant le transport des individus et des biens. Avec la numérisation croissante, les clients recherchent des solutions flexibles et pratiques pour accéder à divers services de location. Ce projet vise à créer une plateforme numérique multiplateforme dédiée à l'administration des services de location de véhicules, permettant à plusieurs agences de gérer efficacement leurs flottes, leurs chauffeurs, et d'offrir leurs services directement aux clients. Inspirée par des plateformes comme Alibaba, cette solution offrira une interface centralisée où les agences pourront s'enregistrer et proposer leurs services dans un environnement sécurisé et intuitif.

Partie I: PHASE D'ANALYSE

I. Présentation générale du projet

Le projet consiste à développer une plateforme web innovante qui permettra à des agences de location de véhicules de s'inscrire et de gérer leurs opérations en ligne. Chaque agence disposera d'un espace dédié pour administrer ses véhicules, affecter des chauffeurs, et proposer ces services à une clientèle diversifiée. La plateforme offrira également aux clients la possibilité de rechercher, comparer, et réserver des véhicules en fonction de leurs besoins, tout en ayant accès à des informations en temps réel sur la disponibilité et les tarifs.

L'objectif principal est de simplifier la gestion des agences tout en créant un point de rencontre dynamique entre l'offre et la demande. En plus d'une gestion des opérations quotidiennes, les agences pourront suivre l'état des véhicules, gérer les réservations, et analyser les performances via des tableaux de bord personnalisés. Du côté client, l'interface permet une navigation fluide et des options de réservation rapide, garantissant une expérience utilisateur optimale.

Ce projet se distingue par sa flexibilité et son évolutivité, permettant à une multitude d'agences de s'inscrire, quelque soit leur taille ou leur région, créant ainsi un écosystème fluide de services de location de véhicules.

II. Description du contexte

Dans une société en constante évolution, où la mobilité joue un rôle essentiel dans la vie quotidienne, l'accès à un moyen de transport rapide, sécurisé et économique devient une nécessité incontournable. L'agence de location de véhicules se positionne comme un acteur clé dans ce domaine, en offrant des solutions flexibles et adaptées aux besoins spécifiques de deux cibles principales : les particuliers ou entreprises cherchant à louer un véhicule, et les propriétaires de véhicules souhaitant rentabiliser leur flotte.

- **Ceux qui souhaitent louer des véhicules** : Cette catégorie regroupe des personnes ou entreprises ayant des besoins temporaires de transport, que ce soit pour des déplacements personnels, des voyages professionnels, ou encore pour faciliter la logistique lors d'événements spéciaux (mariages, séminaires, etc.). Louer un véhicule offre ainsi une flexibilité accrue pour répondre à des besoins ponctuels, que ce soit pour des trajets urbains ou des voyages de longue distance, sans les contraintes d'un achat.
- **Ceux qui désirent mettre en location leurs véhicules** : Cette catégorie inclut non seulement les propriétaires de parcs automobiles, mais aussi les particuliers ayant un véhicule inutilisé ou souhaitant en tirer un revenu supplémentaire. Les agences de location elles-mêmes peuvent également utiliser la plateforme pour sous-louer des véhicules, maximisant ainsi l'utilisation de leur flotte. Que ce soit pour des voitures de tourisme, des utilitaires, ou des véhicules haut de gamme, ce modèle permet une rentabilité accrue et une gestion simplifiée.

Ainsi, la mise sur pied d'une plateforme destinée aux agences de location de véhicules garantit de nombreux avantages tels que :

- **Une gestion centralisée et simplifiée** : La plateforme permet aux agences de suivre et gérer efficacement l'état de leur flotte, les réservations, les contrats de location et les paiements, le tout à partir d'une interface unique.
- **Une visibilité accrue** : En étant présentes sur une plateforme dédiée, les agences augmentent leur visibilité auprès des clients potentiels. Cette présence en ligne leur permet de toucher un public plus large, en particulier

les clients qui recherchent une solution rapide et pratique pour leurs besoins de transport.

- **L'optimisation des revenus** : Les propriétaires de véhicules peuvent rentabiliser leurs biens en les mettant en location lorsque ceux-ci ne sont pas utilisés. Cela contribue à maximiser les profits, tant pour les particuliers que pour les gestionnaires de parcs automobiles.
- **La transparence des transactions** : Grâce à des outils de suivi et de gestion en temps réel, la plateforme assure la transparence des transactions, facilitant ainsi la communication entre les loueurs et les locataires, et réduisant les litiges éventuels.
- **La sécurité et la fiabilité** : La plateforme peut intégrer des systèmes de vérification et de sécurité pour s'assurer que les véhicules loués et les locataires répondent à des critères précis. Cela contribue à renforcer la confiance entre les parties, tout en minimisant les risques liés à la location.
- **Un service client amélioré** : En automatisant certains processus comme la réservation et les paiements, les agences peuvent se concentrer davantage sur l'accompagnement de leurs clients, leur offrant ainsi un service de qualité et personnalisé.

III. Problématique

Dans un contexte où la mobilité est devenue un facteur clé de productivité et de bien-être, la demande pour des solutions de transport souples, sécurisées et accessibles s'intensifie. Toutefois, les agences de location de véhicules font face à plusieurs défis : comment maximiser l'utilisation de leur flotte tout en assurant une gestion fluide et transparente des transactions, ainsi qu'une garantie sur la sécurité de leur flotte ? Comment concilier la nécessité pour les locataires de trouver des véhicules adaptés à leurs besoins ponctuels avec celle des propriétaires de rentabiliser leurs actifs ? De plus, la concurrence accrue dans le secteur de la location de véhicules exige une visibilité optimale pour capter et fidéliser les clients.

Ainsi, la problématique centrale est la suivante : **comment mettre en place une plateforme qui réponde aux attentes de ces deux groupes cibles – locataires et propriétaires de véhicules – tout en garantissant une gestion simplifiée, une optimisation des revenus et une expérience utilisateur fiable et sécurisée ?**

Ce questionnement implique d'adresser des enjeux de logistique, de transparence des transactions, de sécurité, ainsi que de compétitivité sur un marché en constante évolution.

IV. Définition des besoins

A. Besoins fonctionnels

Les besoins fonctionnels d'un projet désignent les exigences et les fonctionnalités que le projet doit remplir pour atteindre ses objectifs. Ils décrivent ce que le projet doit faire, comment il doit fonctionner et quels résultats il doit produire.

Les besoins fonctionnels sont généralement exprimés en termes de : Fonctionnalités, Performances attendues (vitesse, sécurité, fiabilité, etc.), les interactions, données et sécurité.

Les besoins fonctionnels sont généralement identifiés à partir des : Objectifs du projet, Besoins des utilisateurs, Exigences réglementaires et contraintes techniques.

Dans le cas de notre service de car rental, les besoins fonctionnels sont les suivants :

- **Fonctionnalités des clients :**

- Rechercher un véhicule
- Faire une réservation
- Effectuer un paiement
- Annuler une réservation
- Demander un remboursement

- ❖ **Fonctionnalités des Agences**

- Gestion des agences:

- Enregistrer une agence de location et gérer son compte (modifier, supprimer).
- Enregistrer et gérer ses points de services.
- Enregistrer et gérer ses véhicules
- Enregistrer ou supprimer un chauffeur.

- Gestion des véhicules :

- Modifier les informations d'un véhicule.
- Supprimer un véhicule.
- Enregistrement du trafic d'un véhicule.
- Évaluer un véhicule
- Facturer un véhicule

- Gestion des chauffeurs :
 - Ajouter, modifier ou supprimer un chauffeur.
 - Vérifier l'identité d'un chauffeur.
 - Évaluer un chauffeur.
 - Louer un chauffeur ou l'ajouter à une réservation.
- Gestion des locations : Dans notre compréhension, une location est la réservation d'un véhicule qui a été totalement payé.
 - Initier une location
 - Assigner un chauffeur à une location.
 - Facturer une location (par heure et par jour)
 - Annuler une réservation.
- Gestion des finances :
 - Générer les factures d'une location et/ou d'une réservation.
 - Effectuer le remboursement d'un client
 - Effectuer des transactions en utilisant des services de paiement tels que OM, MOMO et banques.
- Statistiques et résultats :
 - Historique du trafic de chaque véhicule.
 - Historique des locations(véhicules, chauffeur, utilisateur concerné, autres informations sur la location)
 - Filtrages dans la recherche
 - Historique des transactions effectuées.
 - Rendu du chiffre d'affaires.
 - Génération d'un compte rendu de l'utilisation de la plateforme.

En résumé, les besoins fonctionnels définissent ce que le projet doit faire pour atteindre ses objectifs et répondre aux besoins des utilisateurs.

B. Besoins non fonctionnels

Les besoins non fonctionnels d'un projet sont des exigences qui ne sont pas directement liées aux fonctionnalités du système, mais qui sont essentielles pour garantir sa qualité, sa performance, sa sécurité et son acceptation par les utilisateurs.

Ils sont souvent exprimés sous forme de contraintes ou d'exigences.

- Le système doit être compatible avec les navigateurs Chrome, Firefox et Safari.
- Le système doit être sûr, protéger les informations des utilisateurs
- Le système doit avoir un bon SEO (search engine optimisation)
- Le système doit prendre en compte l'internationalisation. Les utilisateurs devraient avoir la possibilité de changer de langue par exemple passer de l'anglais au français et du français à l'anglais.
- L'application doit être responsive permettant un affichage des pages en fonction de la taille de l'écran des utilisateurs.
- Le temps de recharge d'une page doit être assez court pour éviter que les utilisateurs se lassent d'attendre.
- Les couleurs choisies pour l'interface graphique doivent être harmonieuses et non agressives pour les yeux.

C. Objectif général

L'objectif général de ce projet est de développer une plateforme numérique responsive qui centralise la gestion des agences de location de véhicules. Elle permettra à ces agences de s'enregistrer, de gérer leurs véhicules et leurs chauffeurs, ainsi que de proposer leurs services directement aux clients. Ce projet vise à améliorer l'efficacité opérationnelle des agences tout en offrant aux clients un accès simplifié à une large gamme d'options de location, garantissant ainsi une meilleure expérience utilisateur pour les deux parties.

V. Résultat attendu

Le projet vise à livrer une plateforme multifonctionnelle qui offrira aux agences de location un large éventail d'outils pour gérer efficacement leurs opérations. Les interactions attendues pour les agences incluent :

- **Gestion des véhicules** : Les agences pourront ajouter, modifier, et supprimer des véhicules dans leur flotte, en renseignant des détails tels que la marque, le modèle, l'année, l'état, et les spécificités techniques.
- **Gestion des chauffeurs** : Chaque agence pourra ajouter des chauffeurs, gérer leurs informations, attribuer des véhicules spécifiques et suivre leurs missions en temps réel.
- **Suivi et traçabilité des véhicules** : Grâce à l'intégration de systèmes GPS, les agences pourront suivre en temps réel la localisation de chaque véhicule, ce qui permettra une meilleure gestion des trajets et des interventions en cas de problème.
- **Historique des trajets** : La plateforme fournira un historique détaillé des trajets effectués par chaque véhicule. Les agences pourront consulter ces informations pour voir les distances parcourues, les itinéraires empruntés, la durée des trajets, ainsi que les arrêts effectués.
- **Gestion des réservations** : Les agences pourront gérer les demandes de réservation, approuver ou refuser des requêtes, et planifier la disponibilité des véhicules en fonction des besoins des clients.
- **Suivi de la maintenance** : Les agences pourront recevoir des alertes sur la maintenance requise pour chaque véhicule (vidange, contrôle technique, changement de pneus, etc.), garantissant ainsi la sécurité et la performance des véhicules.
- **Facturation et paiements** : La plateforme intégrera des outils de facturation automatisée, permettant aux agences de générer des factures pour les clients, de suivre les paiements reçus, et de gérer les transactions via une passerelle de paiement sécurisée.
- **Rapports et tableaux de bord** : Les agences auront accès à des tableaux de bord qui fournissent des rapports sur l'utilisation des véhicules, les revenus générés, le nombre de réservations, et les performances des chauffeurs, ce qui facilitera l'analyse des performances globales.

VI. Démarche d'intervention

A. Architecture(Clean Architecture)

L'utilisation de la **Clean Architecture** pour un système de gestion de location de véhicules multi-agences permet de garantir :

- **Modularité** : chaque composant du système est indépendant et peut être facilement modifié ou remplacé sans affecter les autres parties.
- **Scalabilité** : il est simple d'ajouter de nouvelles fonctionnalités ou services sans modifier la structure existante.
- **Testabilité** : chaque couche peut être testée de manière indépendante, facilitant les tests unitaires et d'intégration.
- **Séparation des responsabilités** : la logique métier, la gestion des données et les interfaces sont distinctes, assurant un code propre et maintenable.

Remarque : Utilisation des design pattern(patron de conception)

1. Entities (Entités)

Cette couche contient les objets métier fondamentaux et les règles d'affaires générales. Elle ne dépend pas des autres couches. Dans ce projet, on peut y trouver :

- **Agence** : propriétés comme nom, adresse, identifiant.
- **Point de service** : propriétés comme nom, adresse, identifiant.
- **Véhicule** : modèle, numéro d'immatriculation, statut (disponible, en location, etc.).
- **Conducteur** : informations personnelles, permis, historique.
- **Location** : date de début/fin, état, montant payé.
- **Client** : identifiants clients, historique de location.

2. Use Cases (Cas d'Utilisation)

Les cas d'utilisation sont responsables de la logique spécifique de l'application. Ils orchestrent les interactions entre les entités pour répondre aux besoins du système. Exemples dans ce projet :

- **Gérer les véhicules** : création, modification, suppression, suivi GPS en temps réel.
- **Enregistrer une agence** : ajouter une nouvelle agence au système.
- **Réservation de véhicules** : gestion des réservations et paiement.
- **Suivi des véhicules** : trace des trajets des véhicules, gestion de geofencing.
- **Rapports** : génération des rapports d'utilisation, d'historique des trajets.

3. Interface Adapters (Adaptateurs d'Interface)

Cette couche convertit les données pour qu'elles puissent être utilisées par les cas d'utilisation. Elle contient les :

- **API REST** : pour l'interaction avec les agences et les clients (inscription, gestion des véhicules, gestion des chauffeurs, location).
- **Repository Pattern** : gestion des interactions avec la base de données (sauvegarde des véhicules, chauffeurs, agences, locations).
- **Adaptateur GPS** : pour récupérer les données de géolocalisation en temps réel.
- **Adaptateurs de paiement** : pour traiter les transactions des clients.

4. Frameworks & Drivers (Cadres et Infrastructure)

Cette couche contient l'infrastructure et les technologies spécifiques au système, comme :

- **Framework Web (spring boot)** : pour gérer les routes HTTP et les contrôleurs.
- **Base de données** : MySQL ou Scylla Db pour la persistance des entités (agences, véhicules, chauffeurs, clients).
- **Système de gestion de fichiers** : pour stocker les documents et images des véhicules.
- **Authentification** : gestion des sessions et permissions via JWT/OAuth2 et Open KYC(pour les conducteurs).

B. Planification

1. Analyse et Conception

i. Analyse des Besoins

- **Objectifs du Projet:** Définir clairement les objectifs du site web, comme la gestion des agences de location, des véhicules, des conducteurs et des clients.
- **Exigences fonctionnelles:** Décrire les fonctionnalités requises telles que l'inscription des agences, la gestion des véhicules, la réservation en ligne, etc.
- **Exigences Non Fonctionnelles:** Définir les aspects techniques comme les performances, la sécurité, et la scalabilité.

ii. Conception

- **Architecture du Système:** Élaborer une architecture globale du système, incluant les modules principaux (gestion des utilisateurs, gestion des véhicules, gestion des réservations).
- **Conception de la Base de Données:** Concevoir le modèle de données en définissant les tables, les relations, et les clés primaires/étrangères.
- **Wireframes et Maquettes:** Créer des wireframes pour les pages principales du site et des maquettes pour le design visuel.

- **Design de l'Interface Utilisateur:** Concevoir l'interface utilisateur en tenant compte de l'expérience utilisateur et de l'accessibilité.
- **Planification des Tests:** Définir une stratégie de test incluant les tests fonctionnels, d'intégration, et de performance.

2. Phase Développement

i. Configuration de l'Environnement

- **Mise en place des Outils de Développement:** Installer les outils nécessaires (IDE, systèmes de gestion de version, etc.).
- **Configuration des Environnements de Développement:** Préparer les environnements de développement, de test, et de production.

ii. Développement des Modules

- **Module de Gestion des Utilisateurs:** Développer les fonctionnalités pour l'inscription, la gestion des profils, et la gestion des rôles.
- **Module de Gestion des Véhicules:** Mettre en place les fonctionnalités pour l'ajout, la modification, et la suppression des véhicules.
- **Module de réservation:** Créer les fonctionnalités permettant aux clients de réserver des véhicules en ligne.
- **Module Administratif:** Développer les outils pour les administrateurs afin de gérer les agences, les réservations, et les rapports.

3. Phase de Développement

i. Préparation au Déploiement

- **Préparation de l'Infrastructure:** Configurer les serveurs et les services nécessaires pour héberger le site web.
- **Migration des Données:** Transférer les données de développement et de

test vers l'environnement de production.

- **Plan de déploiement:** Élaborer un plan de déploiement incluant les étapes de mise en ligne et les procédures de sauvegarde.

ii. Déploiement

- **Déploiement initial:** Mettre en ligne le site web en suivant le plan de déploiement.
- **Configuration Post-Déploiement:** Effectuer les configurations nécessaires après le déploiement initial (paramétrage, optimisation, etc.).
- **Validation en Production:** Assurer que le site fonctionne correctement en production et que toutes les fonctionnalités sont opérationnelles.

VII. Outil collaboratif

Les différents outils que nous utiliserons sont regroupés en fonction du domaine d'activité énuméré comme suit:

- Discussion: Whatsapp, Telegram, google meet
- Modelisation: Online drawIO
- Report: Overleaf et Google Doc
- Implementation: Github / Gitlab
- Administration : Jira

Conclusion

Cette partie d'analyse présente une vue d'ensemble complète du projet de développement d'un site web de gestion de location de véhicules, en détaillant la présentation générale, le contexte, et la problématique identifiée. Nous avons défini les besoins fonctionnels et non fonctionnels, clarifié les objectifs généraux, et précisé les résultats attendus, ce qui établit des bases solides pour la suite du projet. La démarche d'intervention, incluant l'architecture, les méthodes, et la planification, a été soigneusement élaborée pour garantir une exécution cohérente et efficace. L'utilisation d'outils collaboratifs a également été envisagée pour optimiser le travail en équipe. En somme, ce cahier d'analyse fournit une feuille de route claire et structurée, préparant le terrain pour la phase de développement et assurant que le projet est bien aligné avec les attentes et les objectifs définis.

Partie II: PHASE DE CONCEPTION DU PROJET

L'objectif principal de cette partie est de fournir une vision claire et structurée de la conception du système, en définissant les choix technologiques, les modèles de données, les interactions entre les différentes composantes et les contraintes techniques à respecter. Il sert de référence pour les développeurs, les architectes logiciels et les parties prenantes impliquées dans la mise en œuvre du projet.

Dans un premier temps, nous préciserons les principes directeurs qui ont orienté la conception du système, notamment en matière de performances, de sécurité et de maintenabilité. Ensuite, nous détaillerons l'architecture globale, en mettant en évidence les différents modules et leur rôle respectif. Enfin, nous exposerons les principales décisions techniques, y compris les technologies utilisées, les schémas de bases de données, et les mécanismes assurant l'intégrité et la cohérence des données.

Ce document vise ainsi à garantir une compréhension partagée du fonctionnement du système et à faciliter son implémentation tout en assurant son évolutivité et sa robustesse.

I. Architecture du projet

L'architecture du projet définit la structure et l'organisation des composants du système. Elle garantit la modularité, la scalabilité et la maintenabilité. Un bon design architectural optimise les performances et facilite les évolutions.

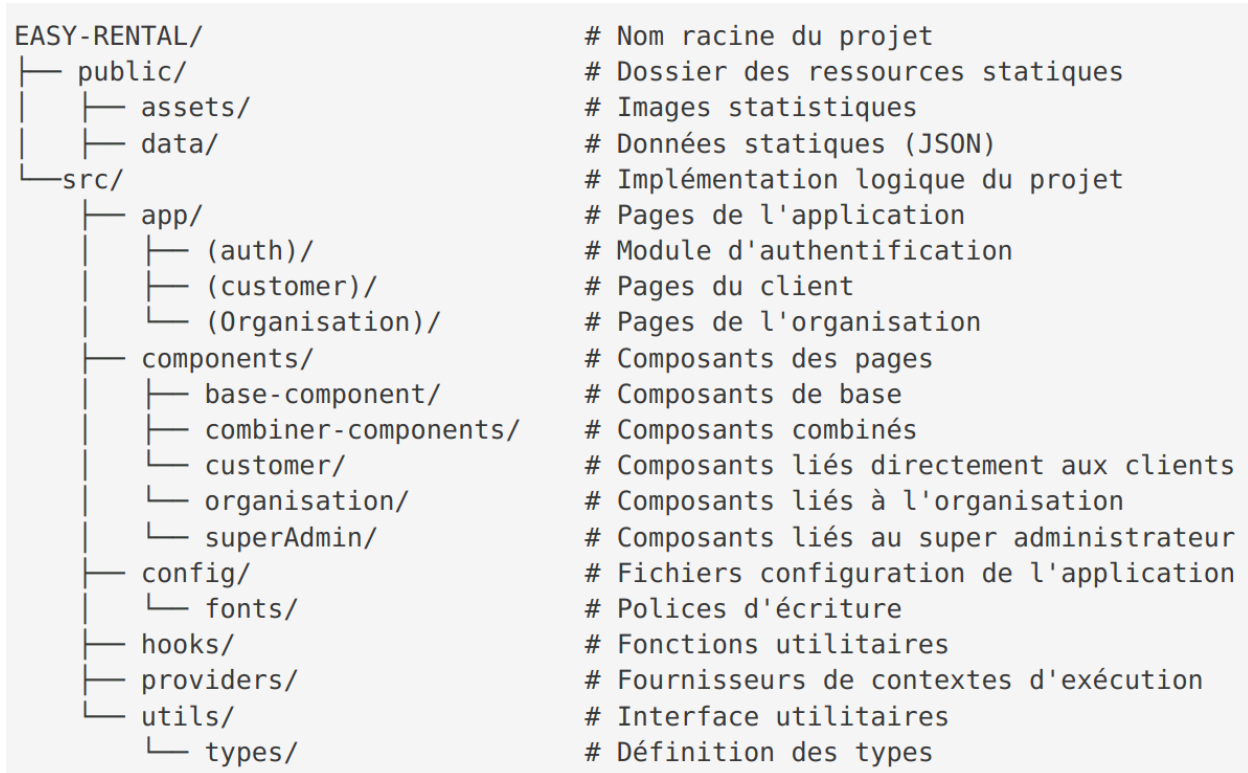


Figure 1: Architecture du projet Next JS

II. Conception fonctionnelle

La conception fonctionnelle décrit le comportement du système selon les besoins utilisateurs. Elle définit les fonctionnalités, les flux d'information et les interactions. Une bonne conception assure clarté, cohérence et efficacité du système.

1. Modélisation détaillée (UML)

a. Diagramme de contexte

Le diagramme de contexte représente le système dans son ensemble et ses interactions avec les acteurs externes. Il permet d'avoir une vue globale des échanges d'informations entre le système et son environnement.

Ci-dessous le schéma :

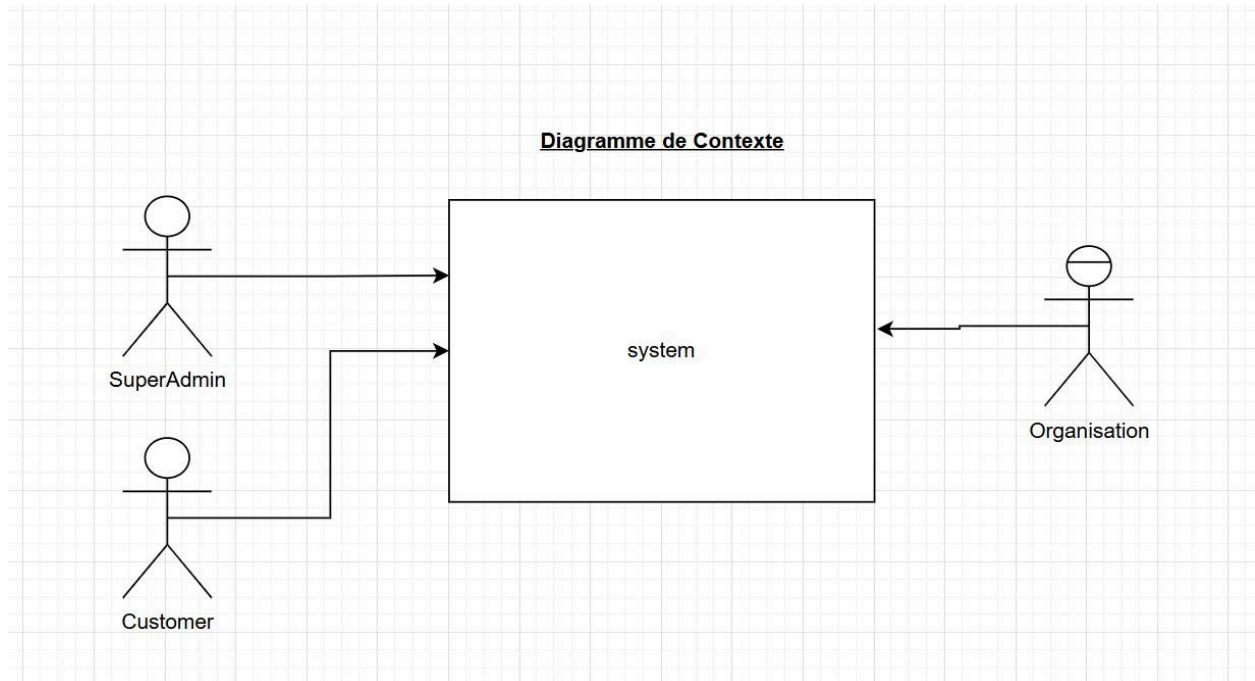


Figure 2: Diagramme de contexte

b. Diagramme de package

Le diagramme de package organise les différentes parties du système en regroupant les classes par modules ou fonctionnalités. Il facilite la compréhension de la structure globale et des dépendances entre les composants.

Ci-dessous le schéma :

c. Diagramme de classes

Le diagramme de classes modélise la structure statique du système en représentant ses objets, leurs attributs, méthodes et relations. Il est essentiel pour définir l'organisation et les interactions entre les entités du projet.

Ci-dessous le schéma :

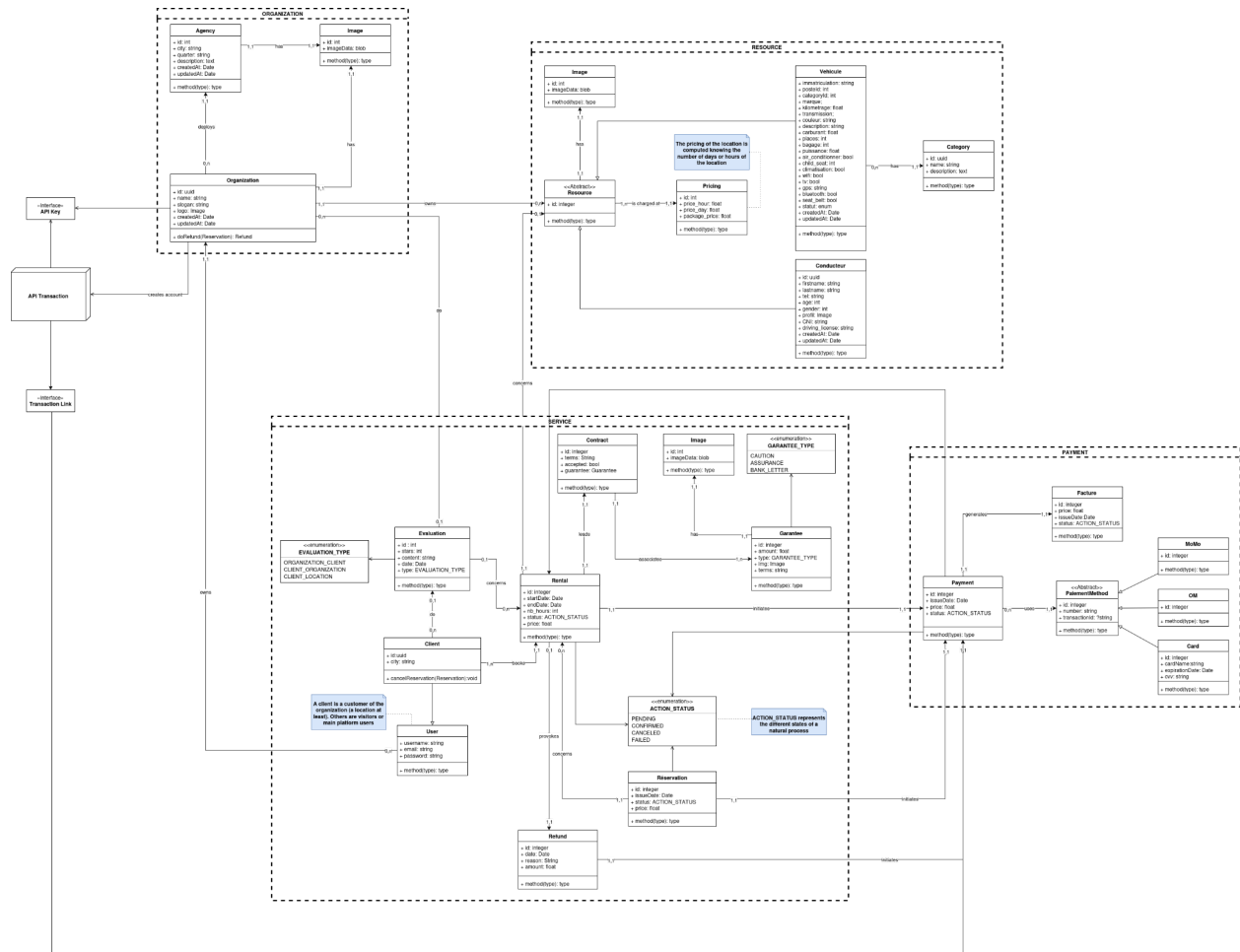


Figure 3: Diagramme de classe

2. Interface utilisateur (UI)

Voir la modélisation réalisée sur figma :

<https://www.figma.com/design/Uxy1pgOXeEguAcelxeMHmk/reseaux?node-id=4-13307&t=BHIRznJdUvtU9O45-1>

III. Conception technique

La conception technique définit les choix d'implémentation du système en fonction des exigences fonctionnelles et non fonctionnelles.

1. Choix des technologies et justifications

Les technologies sélectionnées reposent sur des critères de performance, de scalabilité et de compatibilité avec l'écosystème du projet. Chaque choix est justifié par son adéquation aux besoins métier et techniques.

a. Base de données

La base de données centralise et structure les informations du système. Son organisation garantit l'intégrité, la cohérence et l'optimisation des requêtes pour assurer des performances optimales.

b. Langages de Programmations

Les langages de programmation retenus sont adaptés aux besoins du projet, en tenant compte des performances, de la maintenabilité et de la flexibilité. Ainsi suivent ceux qui ont fait office de choix pour ce projet :

- **Frontend :**

- **HTML**, Hypertext Markup Language est le langage de balisage standard pour les documents conçus pour être affichés dans un navigateur Web.
- **CSS**, Les feuilles de style en cascade sont un langage de feuille de style utilisé pour spécifier la présentation et le style d'un document écrit dans un langage de balisage tel que HTML ou XML.
- **Typescript**, Langage de programmation de haut niveau gratuit et open source développé par Microsoft qui ajoute un typage statique avec des annotations de type facultatives à JavaScript.

- **Backend :**

- **Java**, Langage de programmation de haut niveau, basé sur des classes et orienté objet, conçu pour avoir le moins de dépendances d'implémentation possible.

c. Frameworks

Les frameworks utilisés offrent des outils et des conventions pour accélérer le développement et garantir une architecture robuste. Ainsi suivent ceux qui ont fait office de choix pour ce projet :

- **Frontend :**
 - **Next JS**, Framework de développement Web open source créé par la société privée Vercel fournissant des applications Web basées sur React avec un rendu côté serveur et un rendu statique.
 - **Tailwind CSS**, Framework CSS open source. Contrairement à d'autres frameworks, comme Bootstrap, il ne fournit pas une série de classes prédéfinies pour des éléments tels que des boutons ou des tableaux.
- **Backend :**
 - **Spring Boot**, Framework Java open source utilisé pour programmer des applications Spring autonomes de qualité production avec un ensemble de bibliothèques qui facilitent le démarrage et la gestion des projets.

Partie III: API DE GEOFENCE

1. Introduction

Cette partie présente le développement d'une API de geofencing conçue pour suivre et surveiller les déplacements de véhicules par rapport à des zones géographiques prédéfinies. Ce système permet de générer des alertes lorsqu'un véhicule entre ou sort d'une zone spécifique, offrant ainsi une solution de surveillance en temps réel.

2. Architecture du Système

2.1 Technologies Utilisées

- **Langage de programmation** : Java
- **Framework** : Spring Boot
- **Persistance** : JPA/Hibernate
- **Communication en temps réel** : WebSocket
- **Architecture** : REST API
- **Bibliothèques** : Lombok (pour réduire le code boilerplate)

2.2 Structure du Projet

L'application est organisée selon une architecture en couches classique:

- **Controllers** : Points d'entrée de l'API REST
- **Services** : Logique métier et traitement des données
- **Repositories** : Accès aux données
- **Models** : Entités JPA représentant les données
- **Enums** : Types énumérés pour les rôles et types d'alertes
- **Configuration** : Configuration des composants Spring

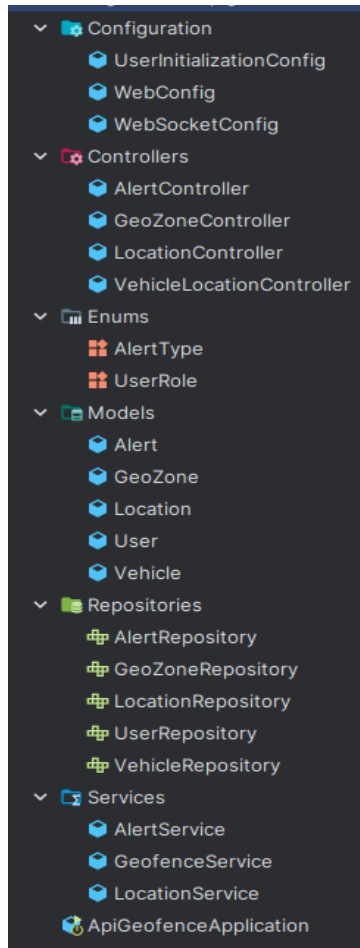


Figure 4: Architecture de l'API de Geofence

3. Modèles de Données

3.1 Entités Principales

- **User** : Utilisateurs du système (rôles: CLIENT, ADMIN)
- **Vehicle** : Véhicules suivis avec leur dernière position connue
- **GeoZone** : Zones géographiques définies par un centre et un rayon
- **Location** : Historique des positions des utilisateurs
- **Alert** : Notifications générées lors d'événements spécifiques

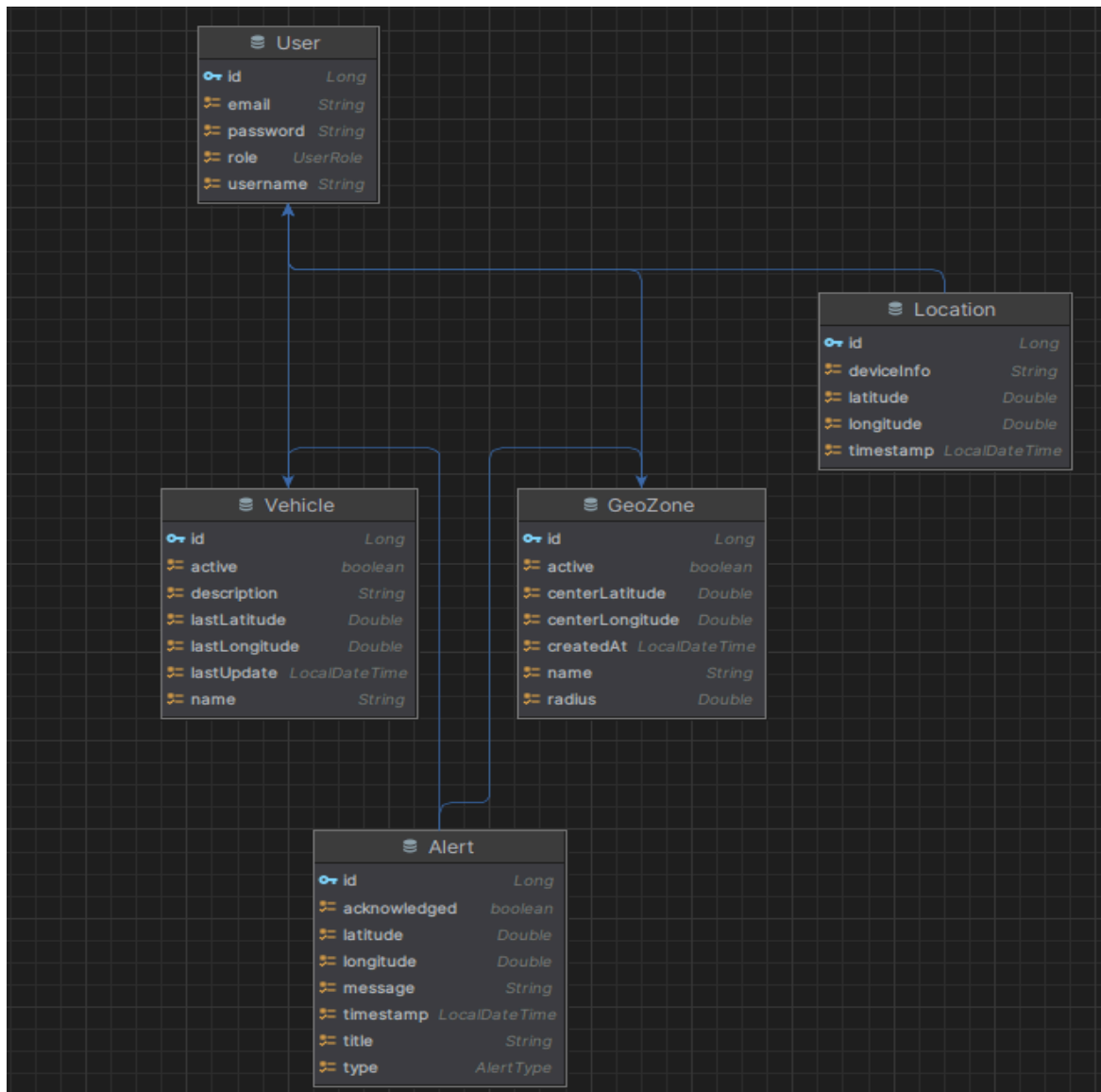


Figure 5: Diagramme de classe - Geofence

4. Fonctionnalités Principales

4.1 Gestion des Zones Géographiques

Le système permet de créer, lire, mettre à jour et supprimer des zones géographiques (opérations CRUD). Chaque zone est définie par:

- Un nom
- Une position centrale (latitude/longitude)
- Un rayon en mètres
- Un statut (actif/inactif)

4.2 Suivi des Véhicules

L'application permet de:

- Mettre à jour la position d'un véhicule
- Vérifier si un véhicule est dans une zone spécifique
- Suivre l'historique des mouvements

4.3 Système d'Alertes

Des alertes sont générées dans les cas suivants:

- Entrée d'un véhicule dans une zone
- Sortie d'un véhicule d'une zone

4.4 Notifications en Temps Réel

Le système utilise WebSocket pour envoyer des notifications en temps réel aux clients connectés lorsqu'une alerte est générée.

5. Diagramme de séquence

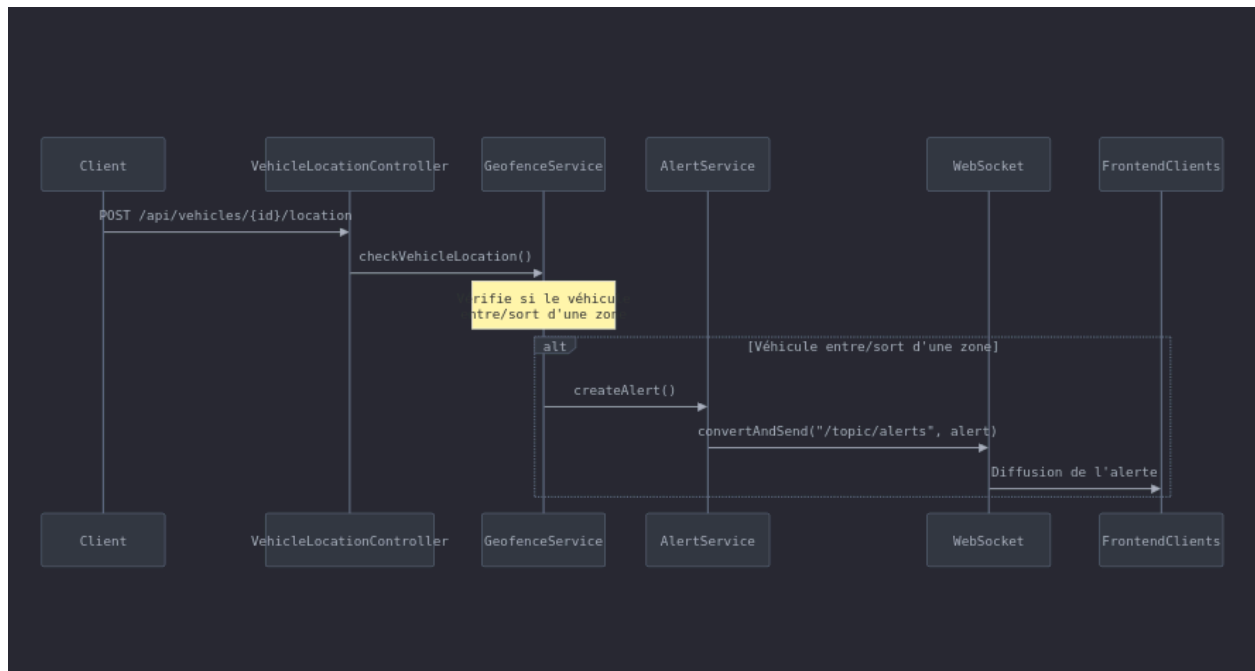


Figure 6: Diagramme de séquence - Geofence

Le diagramme de séquence illustre le flux d'interactions entre les différents composants du système lorsqu'un véhicule entre ou sort d'une zone géographique. Voici l'explication détaillée des interactions :

Acteurs et Composants

- Client : L'application cliente qui envoie les mises à jour de position des véhicules
- VehicleLocationController : Contrôleur REST qui reçoit les requêtes de mise à jour de position
- GeofenceService : Service qui vérifie si un véhicule est entré ou sorti d'une zone
- AlertService : Service responsable de la création et gestion des alertes
- WebSocket : Composant de communication en temps réel
- FrontendClients : Clients frontend qui reçoivent les notifications en temps réel

Flux d'interactions

1. Le processus commence lorsqu'un Client envoie une requête HTTP POST à l'endpoint `/api/vehicles/{id}/location` avec les coordonnées mises à jour du véhicule.
2. Le VehicleLocationController reçoit cette requête et appelle la méthode `checkVehicleLocation()` du GeofenceService.
3. Le GeofenceService vérifie si le véhicule est entré ou sorti d'une zone géographique en comparant sa position précédente et sa position actuelle par rapport aux zones définies.
4. Si un événement d'entrée ou de sortie de zone est détecté, le GeofenceService transmet l'information au AlertService en appelant la méthode `createAlert()`.

5. L'AlertService crée une nouvelle alerte avec les détails appropriés (type d'alerte, position, véhicule, zone concernée, etc.).
6. L'AlertService utilise ensuite le WebSocket pour diffuser cette alerte en temps réel en appelant la méthode `convertAndSend("/topic/alerts", alert)`.
7. Enfin, les FrontendClients connectés au WebSocket reçoivent l'alerte, permettant une notification immédiate aux utilisateurs concernés.

Points importants

- La vérification de position est effectuée pour chaque zone active dans le système.
- Le système génère des alertes distinctes pour les entrées et sorties de zone.
- L'utilisation de WebSocket permet une diffusion en temps réel des alertes à tous les clients frontend concernés.
- Ce mécanisme fonctionne de manière asynchrone, permettant une scalabilité efficace du système.

Ce diagramme met en évidence l'architecture orientée événements du système et illustre comment les différents composants interagissent pour détecter et notifier les mouvements significatifs des véhicules par rapport aux zones géographiques définies.

6. Algorithmes Clés

6.1 Calcul de Distance

Pour déterminer si un véhicule est dans une zone, l'application utilise la formule de **Haversine** pour calculer la distance entre deux points sur la surface terrestre:

```
private double calculateDistance(double lat1, double lon1, double lat2, double lon2) {  
    final int R = 6371000; // Rayon de la Terre en mètres  
  
    double latDistance = Math.toRadians(lat2 - lat1);  
    double lonDistance = Math.toRadians(lon2 - lon1);  
  
    double a = Math.sin(latDistance / 2) * Math.sin(latDistance / 2)  
        + Math.cos(Math.toRadians(lat1)) * Math.cos(Math.toRadians(lat2))  
        * Math.sin(lonDistance / 2) * Math.sin(lonDistance / 2);  
  
    double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));  
  
    return R * c;  
}
```

6.2 Détection de Franchissement de Zone

Le système compare la position précédente et actuelle d'un véhicule par rapport à chaque zone active pour détecter les entrées et sorties:

```
boolean wasInZone = isLocationInZone(  
    vehicle.getLastLatitude(),  
    vehicle.getLastLongitude(),  
    zone  
);  
  
boolean isInZone = isLocationInZone(latitude, longitude, zone);  
  
if (wasInZone && !isInZone) {  
    alertService.createAlert(  
        "Sortie de zone",  
        String.format("Le véhicule %s est sorti de la zone %s",  
            vehicle.getName(), zone.getName()),  
        latitude,  
        longitude,  
        zone  
    );  
}
```

```

        longitude,

        vehicle,

        zone,

        AlertType.ZONE_EXIT

    );
} else if (!wasInZone && isInZone) {
    alertService.createAlert(

        "Entrée en zone",

        String.format("Le véhicule %s est entré dans la zone %s",

            vehicle.getName(), zone.getName()),

        latitude,

        longitude,

        vehicle,

        zone,

        AlertType.ZONE_ENTER

    );
}

```

7. Points Forts et Innovations

7.1 Système d'Alertes Flexible

Le système prend en charge plusieurs types d'alertes et peut être facilement étendu pour inclure de nouveaux types.

7.2 Architecture Temps Réel

Grâce à l'utilisation de WebSocket, les alertes sont transmises instantanément aux clients concernés.

7.3 Gestion des Utilisateurs Multi-rôles

L'application prend en charge différents niveaux d'accès avec des rôles spécifiques (CLIENT, ADMIN).

8. Améliorations Possibles

8.1 Optimisations de Performance

- Mise en cache des zones actives pour réduire les requêtes à la base de données
- Optimisation des calculs de distance pour un grand nombre de zones

8.2 Fonctionnalités Additionnelles

- Support pour les zones polygonales (pas seulement circulaires)
- Prédiction de trajectoire basée sur l'historique des mouvements
- Rapports d'analyse et statistiques d'utilisation

8.3 Sécurité

- Amélioration de l'authentification (actuellement basée sur nom d'utilisateur/mot de passe en clair)
- Mise en place d'autorisations basées sur les rôles pour chaque endpoint

9. Conclusion

Ce projet d'API Geofence démontre une implémentation fonctionnelle d'un système de surveillance de véhicules basé sur des zones géographiques. L'architecture choisie permet une extensibilité facile et des performances satisfaisantes pour un cas d'utilisation standard. Les fonctionnalités de notification en temps réel ajoutent une valeur significative pour les utilisateurs finaux qui peuvent réagir immédiatement aux événements importants.

Partie IV: GUIDE DE DÉPLOIEMENT

I. Introduction

1. Objectif du guide

Ce guide a pour but de fournir des instructions détaillées sur le déploiement de l'application **Easy-Rent**, une plateforme conçue pour permettre aux agences de location de véhicules de proposer leurs services de manière efficace aux clients en ligne. Ce guide couvre toutes les étapes nécessaires à l'installation, à la configuration, au déploiement et à la maintenance de l'application, depuis la mise en place de l'environnement de développement jusqu'à la gestion continue du système après sa mise en production.

L'objectif de ce document est d'expliquer les **étapes précises du déploiement de Easy-Rent sur Vercel**. Puisque l'application a déjà été déployée avec succès via GitHub, ce guide servira à :

- Documenter le **processus de déploiement**, de l'importation du projet à la mise en ligne.
- Expliquer comment **gérer et mettre à jour l'application** sur Vercel après le déploiement initial.
- Détailler les configurations avancées, telles que **les variables d'environnement, les optimisations de performances et la gestion de domaine personnalisé**.

À travers ce guide, vous découvrirez :

- **Comment Easy-Rent a été déployé sur Vercel** en utilisant GitHub.
- **Les étapes à suivre pour effectuer des mises à jour** et assurer une gestion continue de l'application.
- **Comment configurer des options avancées** sur Vercel pour optimiser les performances et la sécurité.
- **Les bonnes pratiques pour maintenir un déploiement efficace** et évolutif.

Les instructions sont spécifiques au déploiement sur la plateforme **Vercel**, en utilisant **GitHub** comme source de code. Ce guide s'adresse à des développeurs et administrateurs ayant déjà des connaissances de base sur **Next.js**, **Tailwind CSS**, **Material UI**, l'utilisation des conteneurs **Docker** et les principes de déploiement sur des environnements cloud comme **Vercel**, **Render** ou **AWS Amplify**.

2. Prérequis :

Avant de suivre ce guide, il est recommandé d'avoir :

- Une connaissance de base de **Next.js** et **Tailwind CSS**.
- Un compte **GitHub**, puisque l'application est déployée depuis un repository GitHub.
- Un compte **Vercel** actif (l'inscription est gratuite sur vercel.com).

3. Gestion de compte et des accès :

Pour faciliter la collaboration et centraliser tous les services nécessaires au développement et au déploiement de **Easy-Rent**, un **compte Google dédié** a été créé. Ce compte est utilisé pour l'inscription sur toutes les plateformes impliquées dans la réalisation et l'hébergement du projet, notamment :

- **GitHub** : Hébergement du code source.
- **Vercel** : Déploiement et hébergement de l'application.
- **Autres outils nécessaires** : Services tiers utiles au projet.

L'utilisation d'un **seul compte** permet à l'équipe de retrouver facilement tous les accès et configurations liés à **Easy-Rent**, évitant ainsi les dispersions et simplifiant la gestion du projet. Le compte google est celui lié à l'adresse email rentalreseau01@gmail.com.

II. Environnement de déploiement

1. Configuration système requise

Avant d'entamer l'installation et le déploiement de l'application, il est crucial de vérifier que le système répond aux exigences minimales. Pour une expérience fluide, il est recommandé de travailler avec des configurations récentes qui peuvent exécuter des applications web modernes avec des outils comme **Next.js** et **Tailwind CSS**.

Les spécifications minimales pour votre système d'exploitation incluent des versions récentes de **Windows**, **macOS**, ou **Linux** (par exemple, Ubuntu 20.04 ou plus récent). Vous aurez également besoin de minimum **4 Go de RAM** et de **10 Go d'espace libre** sur votre disque. Le processeur recommandé est un **Intel i5** ou équivalent pour un meilleur rendement.

2. Pré-requis logiciels

dépendances, bibliothèques ou autres logiciels nécessaires (ex. Java, .NET, bases de données).

Certains logiciels doivent être installés avant de pouvoir travailler avec l'application **Easy-Rent** :

- **Node.js** : Vous aurez besoin d'une version 16 ou plus récente pour exécuter le projet. Node.js est la plateforme de développement pour les applications JavaScript côté serveur et est essentielle pour exécuter **Next.js**.
- **Node Package Manager (npm) ou yarn** : Ces gestionnaires de paquets sont utilisés pour installer les dépendances nécessaires au projet. Vous pouvez choisir entre **npm** (version 7 ou plus) ou **yarn**. Mais de préférence npm pour une compatibilité avec le travail que nous avons utilisé.
- **Git** : Git est utilisé pour gérer le code source et interagir avec **GitHub**, qui héberge le dépôt du projet.

En installant ces outils sur votre machine, vous serez prêt à cloner le dépôt et à commencer l'installation des dépendances.

III. Installation du système

This section outlines the steps to install and configure the **Easy-Rent** application on a production server, focusing on local installation and testing before deployment to **Vercel**. Here's a more detailed explanation of each step:

1. Cloning the GitHub Repository

- **Goal:** Get a copy of the project code on your local machine.
- **How:** Go to the project's GitHub repository (using the link provided) and clone it to your local machine. Cloning creates a local copy of the entire project, including all files and commits, so you can work on it locally.

Command:

```
git clone https://github.com/AdministrationReseau/easy-rental.git
```

- This command fetches the project and stores it in a folder named `easy-rental` (or `easy-rent`, depending on the actual name in the repository).

2. Initializing the Application Locally

- **Goal:** Install the project dependencies required for the application to run.
- **How:** Once you've cloned the repository, you need to navigate into the project folder and install all dependencies listed in the `package.json` file. Dependencies are libraries and tools the application needs to run, such as React, Tailwind CSS, and other utilities.

Command:

```
cd easy-rent  
npm install
```

- - `cd easy-rent` takes you into the project directory.
 - `npm install` reads the `package.json` file, and automatically installs all the required libraries into a folder called `node_modules`.

3. Verifying the Local Setup

- **Goal:** Ensure that the application is working correctly on your local machine before deploying it to production.
- **How:** You will start the development server to test if everything is running as expected. This step allows you to interact with the application on your local machine and see it in action before you deploy it to Vercel.

Command:

npm run dev

- This command will start a local server, and the application will be accessible in your web browser at <http://localhost:3000>.
- If everything is set up correctly, you should see the Easy-Rent application running locally.

Recap

These steps allow you to:

1. Clone the project repository from GitHub.
2. Set up the project locally by installing the necessary dependencies.
3. Test the application locally to ensure that it runs smoothly before deploying it to a server or a platform like Vercel.

Once this is confirmed, you can proceed to the next stages like deploying the application to Vercel for production.

Dans cette section, nous allons détailler les étapes pour installer et configurer l'application **Easy-Rent** sur un serveur de production. Le déploiement se fait principalement via **Vercel**, qui prend en charge la gestion des serveurs.

Suivez les étapes suivantes pour faire l'installation sur votre machine locale

1. **Cloner le dépôt GitHub** : Accédez à GitHub et clonez le dépôt du projet sur votre machine locale.

Commande :

```
git clone https://github.com/AdministrationReseau/easy-rental.git
```

2. **Initialiser l'application en local** : Naviguez dans le répertoire du projet et installez les dépendances :

3. `cd easy-rent`

```
npm install
```

4. **Vérifier le fonctionnement local** : Lancez le serveur local pour tester que tout fonctionne avant de déployer :
npm run dev

L'application sera accessible sur <http://localhost:3000>.

IV. Déploiement de l'application

On a eu à déployer l'application Easy-Rent de deux façons différentes: via un compte vercel

1. Procédure de déploiement

Déployer un projet GitHub sur **Vercel** est assez simple. Les étapes sont les suivant :

1. Créer un compte Vercel

- Aller sur le site de Vercel.
- S'inscrire ou se connecter avec votre compte **GitHub** dans notre cas il s'agit de AdministrationReseau.

2. Importer le projet depuis GitHub

- Cliquez sur "**New Project**".
- Sélectionnez **GitHub** comme plateforme d'importation.
- Autorisez Vercel à accéder à vos repositories.
- Choisissez le projet que vous souhaitez déployer.

3. Configurer le projet

- Vercel détecte automatiquement le framework utilisé (React, Next.js, Vue, etc.).
- Si nécessaire, vous pouvez spécifier les commandes :
 - **Commande d'installation** : `npm install` ou `yarn install`
 - **Commande de build** : `npm run build` ou `yarn build`
 - **Répertoire de sortie** (pour les projets statiques) : `dist` ou `build`
- Ajoutez les variables d'environnement si besoin.

4. Lancer le déploiement

- Cliquez sur "**Deploy**".
- Attendez que Vercel construit et déploie le projet.

5. Obtenir l'URL de déploiement

- Une fois terminé, Vercel vous fournira une URL (ex: `https://mon-projet.vercel.app`).
- Vous pouvez configurer un **nom de domaine personnalisé** dans l'onglet **Domains**.

6. Mise à jour automatique

2. À chaque **push sur GitHub**, Vercel redéploie automatiquement la nouvelle version du projet.

3. Tests de vérification

Pour vérifier que le déploiement est correct, on exécute la commande `npm run build` dans la racine du répertoire du projet.

V. Dépannage (Troubleshooting)

En cas de problème avec le déploiement ou le fonctionnement de **Easy-Rent**, voici les différentes options pour obtenir de l'assistance :

1. Contacter l'équipe Easy-Rent

L'équipe est joignable via l'adresse email du projet : rentalreseau01@gmail.com

2. Assistance GitHub

Si le problème concerne le code source, vous pouvez :

- **Créer une issue sur GitHub** : Accédez au dépôt **Easy-Rent** et ouvrez une nouvelle **issue** pour signaler un problème.
- **Consulter l'historique des commits** pour identifier les modifications récentes pouvant causer des erreurs.

3. Assistance Vercel

Si le problème concerne le déploiement, voici les options disponibles :

- **Documentation officielle** : <https://vercel.com/docs>
- **Forum communautaire** : <https://github.com/vercel/vercel/discussions>
- **Support Vercel** (pour les plans payants) : Contacter l'assistance via <https://vercel.com/help>

4. Autres ressources utiles

- **Next.js Docs** : <https://nextjs.org/docs>
- **Tailwind CSS Docs** : <https://tailwindcss.com/docs>

Conclusion Générale

Ce rapport présente une vue d'ensemble complète du projet de développement d'un site web de gestion de location de véhicules, en détaillant l'analyse, la conception, et le guide de déploiement ainsi que la documentation de l'API de Geofence utilisée. Nous avons défini et traduit les besoins fonctionnels et non fonctionnels, clarifié les objectifs généraux, et précisé les résultats attendus, ce qui établit des bases solides pour la suite du projet. La démarche d'intervention, incluant l'architecture, les méthodes, et la planification, a été soigneusement élaborée pour garantir une exécution cohérente et efficace. Des outils collaboratifs ont également été utilisés pour optimiser le travail en équipe. En somme, ce document fournit un récapitulatif suffisamment structuré, présentant le travail accompli durant ce semestre, assurant que le projet est bien aligné avec les attentes et les objectifs définis.